

CÓMO CLONAR UN REPOSITORIO

EN VISUAL STUDIO CODE

Autor:
Rodrigo CASTILLEJOS

Proyecto:
Control de Dispositivos Embebidos

May 31, 2026

RESUMEN

TABLA DE CONTENIDOS

1	Clonar el repositorio localmente	4
1.1	Prerrequisitos indispensables	4
1.2	Procedimiento detallado paso a paso	4
2	Creación de Ramas Secundarias y Sincronización de Cambios en GitHub	6
2.1	Flujo de trabajo: ¿Cuándo crear la rama secundaria?	6
2.2	Procedimiento gráfico en Visual Studio Code	6
2.3	Procedimiento equivalente mediante la Terminal	7
2.3.1	Análisis del comando de creación de ramas	7
2.3.2	Análisis del comando de preparación de archivos (Stage)	8
2.3.3	Análisis del comando de confirmación de cambios (Commit)	8
2.3.4	Análisis del comando de subida de cambios (Push)	9

1 CLONAR EL REPOSITORIO LOCALMENTE

Este es el método de trabajo estándar, más eficiente y recomendado en la programación cotidiana. Consiste en descargar una copia completa del repositorio (incluyendo todo su historial de cambios, ramas, confirmaciones y metadatos) directamente a tu almacenamiento local (disco duro).

Esto te otorga total autonomía para compilar, probar y modificar los archivos de código de forma local sin requerir una conexión permanente a internet, habilitando además la sincronización directa (envío de *commits* y descarga de actualizaciones) con el servidor remoto de GitHub una vez que recuperes la conexión.

1.1 Prerrequisitos indispensables

Para que este flujo funcione sin problemas, asegúrate de cumplir con lo siguiente antes de proceder:

- Tener instalado la herramienta de control de versiones **Git** en tu sistema operativo Windows.
- Contar con una cuenta activa en **GitHub** y tener iniciada la sesión correspondiente dentro de tu editor Visual Studio Code.

1.2 Procedimiento detallado paso a paso

Sigue atentamente cada uno de los pasos para realizar la clonación:

1. **Obtener la dirección del repositorio en GitHub:** Abre tu navegador de internet y navega hasta la página principal del repositorio que deseas descargar (por ejemplo: [github.com/tu_usuario/Libraries](https://github.com/rodrigo1995/Libraries)).
 - Localiza y haz clic en el botón verde titulado **Code** (ubicado en la parte superior derecha de la sección de archivos).
 - En el menú desplegable que se abre, selecciona la pestaña etiquetada como **HTTPS** (es la más sencilla y estándar).
 - Haz clic en el icono del portapapeles a la derecha del cuadro de texto para copiar la dirección URL de clonación (la cual tendrá una estructura como: `https://github.com/tu_usuario/Libraries.git`), tal como se visualiza en la figura 1.

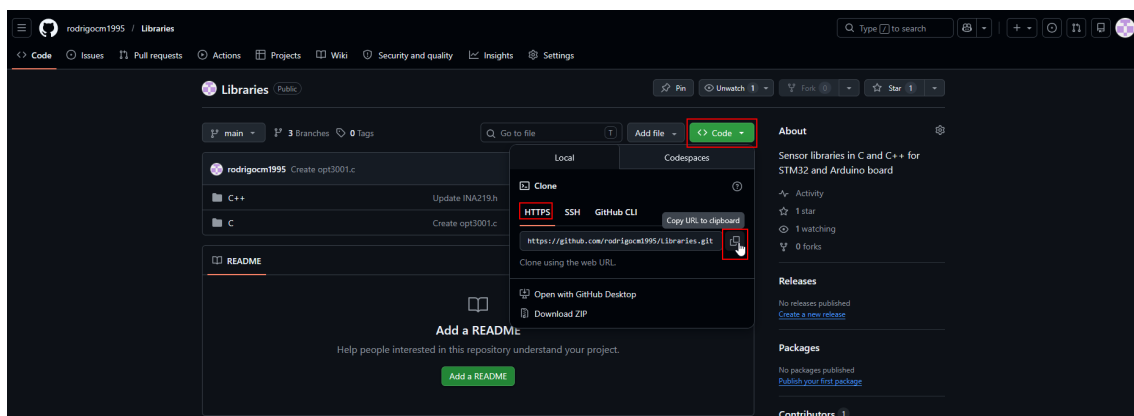


Figura 1: Clonación del repositorio usando la URL web.

2. **Invocar la orden de clonación en Visual Studio Code:** Abre tu editor Visual Studio Code y ejecuta el asistente integrado de Git:

- Abre la paleta de comandos globales presionando la combinación de teclas **Ctrl** + **Shift** + **P**.
 - Escribe la directiva Git: `Clone` y presiona la tecla **Enter**.
 - *Ruta alternativa:* También puedes abrir el panel de **Source Control** (Control de código fuente) en el menú lateral izquierdo (**Ctrl** + **Shift** + **G**) y hacer clic en el botón directo titulado **Clone Repository**.
3. **Introducir la dirección del repositorio:** En la barra de entrada de texto que aparecerá de forma flotante en la parte superior de VS Code, pega la URL que copiaste en el paso 1 y presiona la tecla **Enter** para confirmar.
 4. **Seleccionar el directorio de destino local:** Se abrirá de manera automática una ventana del explorador de archivos de Windows:
 - Navega hasta la carpeta de tu computadora donde sueles organizar tus proyectos de software (por ejemplo, `D:\Proyectos_Programacion`).
 - Haz clic en el botón **Select Repository Location** (Seleccionar ubicación del repositorio).
 - **Nota de organización:** No te preocupes por el orden de los archivos; Visual Studio Code creará de forma automática una subcarpeta con el mismo nombre del repositorio (por ejemplo, una nueva carpeta llamada `Libraries/`) dentro del directorio seleccionado para alojar todo el código de forma independiente.
 5. **Autenticación y Credenciales (Si es requerido):** Si el repositorio es de carácter **privado**, o si es la primera vez que interactúas con GitHub desde el editor en esta máquina:
 - Visual Studio Code te mostrará una notificación flotante pidiendo autorización para conectarse a tu cuenta de GitHub.
 - Haz clic en **Allow** (Permitir) para abrir el navegador web.
 - Inicia sesión en GitHub en tu navegador si no lo has hecho, confirma los permisos haciendo clic en **Authorize Visual-Studio-Code** y permite que el navegador redirija de vuelta a VS Code.
 6. **Cargar el proyecto en el espacio de trabajo activo:** Una vez finalizada la transferencia y descarga de archivos, aparecerá un recuadro de notificación en la esquina inferior derecha con la leyenda *"Would you like to open the cloned repository?"*. Haz clic sobre el botón **Open** (Abrir) para cargar de inmediato los archivos clonados en tu explorador.

Verificación de la clonación correcta

Puedes cerciorarte de que la clonación local y vinculación de Git se efectuaron de manera exitosa observando tres indicadores visuales clave en el editor:

1. El explorador de archivos lateral izquierdo (**Ctrl** + **Shift** + **E**) te mostrará la estructura completa de carpetas (por ejemplo: `C++` y `C`).
2. En el extremo izquierdo de la **barra de estado inferior** del editor, observarás el nombre de la rama actual (típicamente `main` o `master`), acompañada del icono de una bifurcación de Git.
3. Al dirigirte al panel de **Source Control** (**Ctrl** + **Shift** + **G**), verás la lista de cambios vacía y lista para registrar tus próximas modificaciones de código sin necesidad de inicializaciones manuales extra.

2 CREACIÓN DE RAMAS SECUNDARIAS Y SINCRONIZACIÓN DE CAMBIOS EN GITHUB

Una de las mejores prácticas fundamentales en el control de versiones con Git es evitar subir modificaciones directamente a la rama principal (típicamente llamada `main` o `master`), especialmente cuando se trabaja en la actualización o creación de nuevas funciones para bibliotecas (como los archivos `C ▶ INA236 ▶ INA236.c` y `C ▶ INA236 ▶ INA236.h`, por poner un ejemplo). En su lugar, se utilizan ramas secundarias o de desarrollo (*feature branches*), las cuales se fusionan posteriormente tras una revisión.

2.1 Flujo de trabajo: ¿Cuándo crear la rama secundaria?

La directiva estándar es crear la rama secundaria **antes de iniciar cualquier modificación** física sobre los archivos. Esto garantiza que tu espacio de trabajo local esté asignado al objetivo correcto desde el primer commit.

Sin embargo, si has realizado modificaciones en tu código y olvidaste crear la rama de antemano, Git es sumamente flexible: los cambios locales que aún no hayan sido confirmados (*uncommitted changes*) se trasladarán de forma automática contigo en el momento en que crees y te cambies a la nueva rama.

2.2 Procedimiento gráfico en Visual Studio Code

Visual Studio Code proporciona una interfaz visual intuitiva para realizar operaciones de Git sin necesidad de abrir una terminal de comandos. El procedimiento completo se describe a continuación:

1. Crear y activar la rama secundaria:

- Dirígete al extremo izquierdo de la **barra de estado inferior** de Visual Studio Code, donde se indica el nombre de la rama actual (por ejemplo, `main`).
- Haz clic sobre el nombre de la rama activa para desplegar el menú de selección superior.
- Elige la opción **Create new branch...** (Crear nueva rama...).
- Escribe un nombre descriptivo para tu rama (por ejemplo: `desarrollo-ina236` o `feature/update-ina236`) y presiona la tecla `[Enter]`. Observarás que el indicador de rama en la barra de estado inferior se actualiza de forma inmediata.

2. Realizar las modificaciones y guardar:

- Abre los archivos de la biblioteca (en este caso, `C ▶ INA236 ▶ INA236.c` y `C ▶ INA236 ▶ INA236.h`) en el editor.
- Aplica los cambios correspondientes en el código fuente.
- Guarda las ediciones presionando la combinación de teclas `[Ctrl] + [S]`.

3. Confirmar los cambios localmente (Commit):

- Dirígete al panel de **Source Control** (Control de código fuente) en el menú lateral izquierdo (`[Ctrl] + [Shift] + [G]`).
- Identifica tus archivos editados bajo la sección de cambios (**Changes**). Pasa el cursor sobre la palabra *Changes* y haz clic en el icono de suma `[+]` (*Stage All Changes*) para preparar todos los archivos.
- Introduce un comentario breve y explicativo en la caja de texto superior etiquetada como *Message* (por ejemplo: Actualización de funciones en biblioteca INA236).
- Haz clic en el botón azul ****Commit**** (o presiona la combinación `[Ctrl] + [Enter]`) para registrar el cambio local.

4. Subir la rama al servidor remoto (Push):

- Dado que la nueva rama se ha creado inicialmente solo en tu equipo, debes publicarla en el servidor de GitHub.
- Haz clic en el botón azul titulado **Publish Branch** (Publicar rama) que aparece en el panel lateral de Source Control.
- Visual Studio Code se autenticará con tus credenciales de GitHub (solicitando confirmación en el navegador web si es la primera vez que realizas este proceso) y subirá la rama remota de manera transparente.

2.3 Procedimiento equivalente mediante la Terminal

Si prefieres utilizar la terminal integrada de Visual Studio Code (**Ctrl** + **~**), puedes conseguir exactamente el mismo resultado ejecutando la siguiente secuencia de comandos de Git:

```

1      # 1. Crear y cambiar a la rama secundaria de desarrollo
2      git checkout -b desarrollo-ina236
3
4      # [Realiza las modificaciones necesarias en tus archivos y guardalos]
5
6      # 2. Agregar los archivos INA236 modificados al area de preparacion (Stage)
7      git add C/INA236/INA236.c C/INA236/INA236.h
8
9      # 3. Registrar el commit localmente con un mensaje descriptivo
10     git commit -m "Actualizacion de funciones en biblioteca INA236"
11
12     # 4. Subir la rama y establecer la conexion de seguimiento (upstream) con GitHub
13     git push -u origin desarrollo-ina236

```

El Flujo del Pull Request en GitHub

Una vez que hayas subido la rama secundaria de forma exitosa (ya sea por el botón **Publish Branch** o por comando en la terminal), si accedes a la página web de tu repositorio en GitHub, observarás un banner interactivo que te invita a crear un **Pull Request** (Petición de extracción).

Esta funcionalidad web te permite contrastar los cambios de tu rama secundaria con la rama principal **main**, facilitando la revisión detallada del código antes de autorizar su integración definitiva.

2.3.1 Análisis del comando de creación de ramas

El comando abreviado empleado para la inicialización y migración de la rama de trabajo es el siguiente:

```
git checkout -b feature/update-INA236
```

Esta instrucción condensa dos operaciones independientes de Git en una sola línea de comandos. Su desglose sintáctico y funcional se describe a continuación:

git: Invoca el motor del sistema de control de versiones Git en el sistema operativo.

checkout: Le indica a Git que quieres cambiar de contexto o moverte (en este caso, cambiar de rama).

-b: Parámetro (*flag*) que ordena la creación de una nueva rama. Le indica a Git que la rama a la que se desea conmutar no existe previamente en el repositorio local y debe ser inicializada en ese instante.

desarrollo-ina236: Nombre de cadena asignado a la nueva rama secundaria para tus modificaciones.

Bajo el capó, ejecutar esta instrucción abreviada equivale exactamente a realizar por separado los siguientes dos comandos tradicionales de Git:

```
1 # 1. Crear la rama local a partir del estado de la rama activa actual
2 git branch desarrollo-ina236
3 # 2. Desplazar el area de trabajo activa hacia la rama recién creada
4 git checkout desarrollo-ina236
```

2.3.2 Análisis del comando de preparación de archivos (Stage)

El comando empleado para colocar las modificaciones de la biblioteca en el área de preparación es el siguiente:

```
1 git add C/INA236/INA236.c C/INA236/INA236.h
```

Esta instrucción registra de forma selectiva los cambios locales para ser incluidos en la próxima confirmación histórica. Su desglose sintáctico es el siguiente:

git: Invoca la herramienta del sistema de control de versiones Git.

add: Comando de Git encargado de indexar y trasladar los cambios del directorio de trabajo local hacia el ****Área de Preparación**** (*Staging Area*).

c/ina236/ina236.c c/ina236/ina236.h: Rutas relativas a los archivos específicos de la biblioteca del sensor que se desean preparar. Al listarlos de forma explícita y separados por un espacio, Git los seleccionará de forma exclusiva.

El Área de Preparación (Staging Area)

El *Staging Area* funciona como un borrador o sala de espera intermedia en Git. Permite estructurar confirmaciones atómicas y organizadas. Si realizas cambios en múltiples archivos del proyecto, pero solo deseas guardar en el historial la actualización de la biblioteca del sensor INA236, puedes ejecutar `git add` únicamente sobre estos archivos. Cualquier otra modificación permanecerá intacta en tu espacio de trabajo local pero quedará excluida del siguiente *commit*.

2.3.3 Análisis del comando de confirmación de cambios (Commit)

El comando utilizado para consolidar y guardar de manera permanente las modificaciones en el historial local del proyecto es el siguiente:

```
1 git commit -m "Actualizacion de funciones en biblioteca INA236"
```

Esta instrucción crea una instantánea (*snapshot*) o punto de restauración de todos los archivos que fueron indexados en el área de preparación. Su desglose sintáctico se explica a continuación:

git: Invoca la herramienta del sistema de control de versiones Git.

commit: Acción que guarda y empaqueta las modificaciones preparadas en el repositorio local. Cada commit genera un identificador único (hash SHA-1) para rastrear el cambio.

-m: Bandera de mensaje (*message*). Permite añadir una descripción breve al commit directamente desde la terminal, evitando abrir un editor de texto interactivo del sistema.

"actualizacion...": Cadena de texto descriptiva. Es una buena práctica escribir mensajes claros en tiempo presente que sintetizen los cambios introducidos.

2.3.4 *Análisis del comando de subida de cambios (Push)*

El comando utilizado para subir tus confirmaciones locales al repositorio en la nube de GitHub es el siguiente:

```
1 git push -u origin feature/desarrollo-INA26
```

Esta instrucción transfiere los commits guardados localmente hacia el servidor remoto de almacenamiento. Su desglose sintáctico se detalla a continuación:

git: Invoca la herramienta del sistema de control de versiones Git.

push: Comando encargado de empujar o enviar las confirmaciones locales de la rama activa hacia el servidor remoto.

-u: Bandera de vinculación de rama origen (*set upstream*). Establece un vínculo permanente entre tu rama local y la rama remota. Esto te permite usar simplemente `git push` o `git pull` en el futuro, sin necesidad de especificar el servidor ni el nombre de la rama.

origin: Nombre o alias por defecto asignado a la dirección del servidor remoto desde el cual clonaste el proyecto (en este caso, tu repositorio en GitHub).

feature/desarrollo-ina26: Nombre de la rama específica que deseas subir al servidor.